COMPREHENSIVE CODE AUDIT REPORT

SECURITY & QUALITY ASSESSMENT

In-depth security and code quality analysis of three projects: lab2.zip (ALO Labs Command Center), lab.zip (CRM Pro), and alo-labs-avatar-video-studio-ai-agent.zip (ALO Labs Monorepo). This report identifies critical vulnerabilities, architectural issues, and provides prioritized remediation guidance.

Prepared by Z.ai Code Auditor | July 2025

Table of Contents

Table of Contents 1

1. Executive Summary 2

2. Project 1: lab2.zip - ALO Labs Command Center 3

2.1 Project Overview . . . 3

2.2 Critical Findings . . . 3

2.3 High Severity Findings . . . 3

2.4 Medium Severity Findings . . . 4

2.5 Low Severity Findings . . . 4

3. Project 2: lab.zip - CRM Pro 5

3.1 Project Overview . . . 5

3.2 Critical Findings . . . 5

3.3 High Severity Findings . . . 5

3.4 Medium Severity Findings . . . 6

4. Project 3: alo-labs-avatar-video-studio-ai-agent.zip - ALO Labs Monorepo 7

4.1 Project Overview . . . 7

4.2 Critical Findings . . . 7

4.3 High Severity Findings . . . 7

4.4 Medium Severity Findings . . . 8

5. Cross-Project Analysis 9

5.1 Common Vulnerability Patterns . . . 9

5.2 Architecture Comparison . . . 9

5.3 Aggregate Risk Metrics . . . 9

6. Prioritized Remediation Roadmap 11

Phase 1 - Immediate (Week 1) . . . 11

Phase 2 - This Sprint (Weeks 2–3) . . . 11

Phase 3 - Next Sprint (Weeks 4–5) . . . 11

Phase 4 - Ongoing . . . 12

7. Conclusion 12

1. Executive Summary

This report presents a comprehensive security and code quality audit of three software projects developed under the ALO Labs umbrella. The audit was conducted in July 2025 and covers source code analysis, architectural review, dependency assessment, and security vulnerability identification. All three projects were found to be in early prototype or development stages with significant security gaps that must be addressed before any production deployment.

The three audited projects are: **lab2.zip** (ALO Labs Command Center), **lab.zip** (CRM Pro), and **alo-labs-avatar-video-studio-ai-agent.zip** (ALO Labs Monorepo). Across all three codebases, the audit identified **11 Critical**, **21 High**, **30 Medium**, and multiple Low severity findings.

Project	Type	Risk Score	Critical	High	Medium
lab2.zip	Next.js 16 Command Center	**7.0 / 10**	**3**	**7**	**8**
lab.zip	Next.js 16 CRM Pro	**7.5 / 10**	**4**	**7**	**12**
alo-labs-monorepo	Vite+Express Monorepo	**8.2 / 10**	**4**	**7**	**10**

2. Project 1: lab2.zip - ALO Labs Command Center

2.1 Project Overview

lab2.zip contains a **Next.js 16** application serving as the ALO Labs Command Center - a multi-tenant AI command center with 13 pages and 107 source files. The application features AI agent management, clearance-level based access control (client-side only), Prisma ORM integration, and Tailwind CSS styling. The project demonstrates significant feature scope but critically lacks server-side security controls.

2.2 Critical Findings

ID	Finding	Severity	File(s)
C-01	Unauthenticated LLM Proxy - /api/agentos endpoint invokes paid LLM services with zero authentication. Any network-accessible user can send requests and incur costs.	CRITICAL	app/api/agentos/route.ts
C-02	No Authentication Middleware - no middleware.ts exists. Authentication is implemented as a client-side boolean toggle only, trivially bypassable.	CRITICAL	(missing) middleware.ts
C-03	.env file with database URL committed to the project directory, exposing internal database connection details.	CRITICAL	.env

2.3 High Severity Findings

ID	Finding	Severity	File(s)
H-01	TypeScript build errors are silently ignored via ignoreBuildErrors: true in next.config.js, masking real type errors.	HIGH	next.config.mjs
H-02	React Strict Mode is disabled (reactStrictMode: false), removing development-time warnings and double-rendering checks.	HIGH	next.config.mjs
H-03	noImplicitAny: false in tsconfig.json allows implicit any types, defeating TypeScript's type safety.	HIGH	tsconfig.json
H-04	Prisma Client is instantiated without a corresponding schema.prisma file, suggesting incomplete or broken database setup.	HIGH	lib/prisma.ts
H-05	Conflicting type definitions for ClearanceLevel - defined in multiple files with incompatible values.	HIGH	types/, lib/auth-context.tsx
H-06	Entire components/alo/modules/ directory is dead code - 14 files with broken imports that cannot resolve.	HIGH	components/alo/modules/

ID	Finding	Severity	File(s)
H-07	Dynamic Tailwind classes (text-${n.tone}-500) will be purged at build time, causing runtime styling failures.	HIGH	components/agent/AgentCard.tsx

2.4 Medium Severity Findings

ID	Finding	Severity	File(s)
M-01	Fake authentication - login is purely client-side with a setTimeout to simulate delay; no server-side verification.	MEDIUM	lib/auth-context.tsx
M-02	Raw LLM output is returned directly to the client without sanitization or validation, enabling potential injection.	MEDIUM	app/api/agentos/route.ts
M-03	No input length validation on the Agentos API - arbitrarily large payloads can be sent to the LLM endpoint.	MEDIUM	app/api/agentos/route.ts
M-04	No rate limiting on any API endpoint, allowing abuse and potential denial-of-service.	MEDIUM	app/api/
M-05	15+ unused dependencies including dnd-kit, react-hook-form, next-auth, next-intl, increasing attack surface.	MEDIUM	package.json
M-06	Duplicate Toaster components exist in the codebase with slightly different implementations.	MEDIUM	components/ui/toaster.tsx
M-07	Inline style tag embedded directly in the create-agent page component instead of using CSS modules.	MEDIUM	app/(dashboard)/create-agent/page.tsx
M-08	Hardcoded server IP address appears in multiple locations, preventing environment-based configuration.	MEDIUM	Multiple files

2.5 Low Severity Findings

Additional low-severity findings include: inconsistent file naming conventions, missing JSDoc comments on public APIs, inconsistent use of default exports vs named exports, large component files that should be split (several exceeding 300 lines), console.log statements left in production code, hardcoded color values instead of using the design token system, and missing loading/error states on several pages.

3. Project 2: lab.zip - CRM Pro

3.1 Project Overview

lab.zip contains a **Next.js 16** CRM application with a separate Bun mini-service running on port 3001. The project includes 13 CRM panel components and uses Prisma as its ORM. The frontend makes API calls to /api/crm endpoints, but critical route handlers are missing, rendering core functionality non-functional in standalone deployment. The application has severe security deficiencies including wildcard CORS, no authentication, and raw SQL injection vectors.

3.2 Critical Findings

ID	Finding	Severity	File(s)
C-01	Missing /api/crm route - the frontend calls /api/crm?r=... but no corresponding API route handler exists, making the app non-functional standalone.	CRITICAL	app/api/crm/ (missing)
C-02	Zero authentication or authorization on all endpoints - any user can perform full CRUD operations on all data.	CRITICAL	app/api/
C-03	Wildcard CORS configuration (Access-Control-Allow-Origin: *) allows any origin to make requests to the API.	CRITICAL	server.ts / Caddyfile
C-04	Internal error details are leaked to clients - raw error objects and stack traces are returned in 500 responses.	CRITICAL	Multiple API routes

3.3 High Severity Findings

ID	Finding	Severity	File(s)
H-01	Broken CrmProvider component - permanently stuck on "Loading..." state due to missing API endpoint, blocking all CRM functionality.	HIGH	components/CrmProvider.tsx
H-02	$executeRawUnsafe with table name interpolation - user-controlled input flows directly into SQL queries, enabling SQL injection.	HIGH	lib/db.ts
H-03	Caddyfile SSRF vulnerability via XTransformPort query parameter - allows server-side requests to arbitrary ports.	HIGH	Caddyfile
H-04	Stale Agent API route references a non-existent Prisma model, causing runtime errors.	HIGH	app/api/agent/route.ts
H-05	Massive dead code in app-store.ts - 164 lines of unused store definitions.	HIGH	stores/app-store.ts
H-06	No input validation on any API endpoint - all data is accepted and processed without schema validation.	HIGH	app/api/

ID	Finding	Severity	File(s)
H-07	TypeScript strictness and ESLint are effectively disabled - noStrict, no unused checks, no lint enforcement.	HIGH	tsconfig.json, .eslintrc

3.4 Medium Severity Findings

ID	Finding	Severity	File(s)
M-01	15+ unused npm dependencies inflate bundle size and attack surface.	MEDIUM	package.json
M-02	Prisma schema design issues - no unique constraints, tags stored as String instead of Json, no onDelete cascade.	MEDIUM	prisma/schema.prisma
M-03	Inconsistent DATABASE_URL paths across configuration files.	MEDIUM	.env, docker-compose.yml
M-04	Excessive data fetching for dropdown selectors - entire tables loaded into memory for simple select fields.	MEDIUM	Multiple components
M-05	Dashboard displays random data for missing revenue figures using Math.random().	MEDIUM	components/Dashboard.tsx
M-06	Notes pinning feature is in-memory only - pinned state is lost on page refresh.	MEDIUM	components/NotesPanel.tsx
M-07	Settings are not persisted - all configuration changes are lost when the application restarts.	MEDIUM	components/SettingsPanel.tsx
M-08	Toast remove delay is set to 1,000,000ms (11.5 days) - notification auto-dismiss is effectively broken.	MEDIUM	hooks/use-toast.ts
M-09	use-toast hook has a useEffect dependency bug causing stale closure references.	MEDIUM	hooks/use-toast.ts
M-10	getStatusCounts() recomputes on every render instead of being memoized.	MEDIUM	components/CrmProvider.tsx
M-11	No AbortController usage - rapid navigation causes race conditions with stale data overwriting fresh data.	MEDIUM	Multiple components
M-12	seed.mjs references non-existent models - 67KB of dead seed code.	MEDIUM	prisma/seed.mjs

4. Project 3: alo-labs-avatar-video-studio-ai-agent.zip - ALO Labs Monorepo

4.1 Project Overview

This project is a monorepo containing a **Vite+React** frontend, an **Express.js** backend with a 1,131-line server.ts file, and a **Python/FastAPI** sub-project. The repository consists of 37 files total. It uses JSON files as its "database" and deploys via Docker Compose. The project has the highest risk score among the three audited codebases due to the concentration of hardcoded credentials, the complete absence of authentication on ~35 routes, and the use of insecure token generation.

4.2 Critical Findings

ID	Finding	Severity	File(s)
C-01	No authentication or authorization on ANY endpoint - approximately 35 routes are completely unprotected, exposing all data and operations.	CRITICAL	server.ts, main.py
C-02	Hardcoded production credentials in docker-compose.yml - 6 plaintext passwords for database, Redis, and other services.	CRITICAL	docker-compose.yml
C-03	Hardcoded password "admin" with the UI displaying a hint showing this password to the user.	CRITICAL	server.ts, App.tsx
C-04	Plaintext passwords stored in SQL seed data - default user password "admin123" inserted in plain text.	CRITICAL	init.sql
C-05	Login endpoint ignores the password field entirely - authentication only checks if the email exists, accepting any password.	CRITICAL	server.ts

4.3 High Severity Findings

ID	Finding	Severity	File(s)
H-01	Insecure session token generation using Math.random() instead of crypto.randomBytes() - predictable and forgeable tokens.	HIGH	server.ts
H-02	No CORS configuration on Express; wildcard CORS with credentials on FastAPI - both are insecure configurations.	HIGH	server.ts, main.py
H-03	No security headers - no Helmet middleware, no Content-Security-Policy, no X-Frame-Options.	HIGH	server.ts
H-04	XSS vulnerability via innerHTML in CRM HTML template - user input rendered directly as HTML.	HIGH	App.tsx

ID	Finding	Severity	File(s)
H-05	No input validation on any endpoint - all user input is accepted without sanitization or schema validation.	HIGH	server.ts, main.py
H-06	JSON file used as "database" with synchronous read/write operations - prone to data corruption and race conditions.	HIGH	server.ts (saveDb)
H-07	All database ports (PostgreSQL, Redis, etc.) exposed to 0.0.0.0 in docker-compose.yml.	HIGH	docker-compose.yml

4.4 Medium Severity Findings

ID	Finding	Severity	File(s)
M-01	7,938-line monolithic App.tsx component - extremely difficult to maintain, test, or review.	MEDIUM	src/App.tsx
M-02	No TypeScript strict mode enabled; pervasive use of any types throughout the codebase.	MEDIUM	tsconfig.json
M-03	Missing Dockerfiles referenced in docker-compose.yml - the containers cannot be built as configured.	MEDIUM	docker-compose.yml
M-04	No requirements.txt for the Python backend - dependencies are not documented or reproducible.	MEDIUM	python/
M-05	Significant mismatch between API schema/documentation and actual implementation.	MEDIUM	docs/, server.ts
M-06	Real PII (email addresses, phone numbers) hardcoded in source code and seed data.	MEDIUM	Multiple files
M-07	AI prompt injection vulnerability - user input is directly interpolated into system prompts sent to LLMs.	MEDIUM	server.ts
M-08	Synchronous file I/O (fs.readFileSync/writeFileSync) blocking the Node.js event loop.	MEDIUM	server.ts
M-09	No error handling on saveDb() - database write failures are silently ignored.	MEDIUM	server.ts
M-10	No logging framework - all logging uses console.log with no structured output.	MEDIUM	server.ts

5. Cross-Project Analysis

5.1 Common Vulnerability Patterns

The audit reveals systemic issues that span all three projects. These patterns suggest a common development approach - likely AI-assisted code generation - where security was treated as an afterthought rather than a foundational requirement. The following vulnerabilities are present across the portfolio:

Zero Authentication: All 3 projects have no server-side authentication. Access control, where it exists, is purely client-side and trivially bypassable.

Wildcard CORS: 2 of 3 projects use Access-Control-Allow-Origin: *, and the third has no CORS configuration at all.

Hardcoded Credentials: 2 of 3 projects contain plaintext passwords in source code, configuration files, and Docker Compose files.

TypeScript Disabled: All Next.js projects have TypeScript strictness disabled (noImplicitAny: false, ignoreBuildErrors: true).

No Input Validation: No project implements server-side input validation on any endpoint - all user input is processed without sanitization.

Unused Dependencies: All projects contain 15+ unused npm packages, increasing bundle size and potential attack surface.

5.2 Architecture Comparison

Aspect	lab2.zip	lab.zip	alo-labs
Framework	Next.js 16	Next.js 16	Vite + Express
Language	TypeScript	TypeScript	TypeScript + Python
Database	Prisma/PostgreSQL	Prisma/PostgreSQL	JSON files
Authentication	Client-side only	None	Broken (ignores password)
Source Files	107	~90	37
Routing	App Router (13 pages)	App Router + Bun srv	Express (~35 routes)
Containerization	Docker Compose	Docker + Caddy	Docker Compose
Input Validation	None	None	None
Testing	None	None	None

5.3 Aggregate Risk Metrics

Severity	lab2.zip	lab.zip	alo-labs	Total
CRITICAL	3	4	5	**12**
HIGH	7	7	7	**21**
MEDIUM	8	12	10	**30**
LOW	5+	5+	3+	**13+**

6. Prioritized Remediation Roadmap

The following phased remediation plan addresses findings in order of risk impact. Each phase builds upon the previous one, ensuring that the most critical vulnerabilities are resolved first.

Phase 1 - Immediate (Week 1)

Objective: Eliminate all Critical vulnerabilities that expose data and services to unauthorized access.

· **Implement authentication middleware** - Add NextAuth.js or similar auth library with proper session management to all three projects. Create middleware.ts for route protection.

· **Fix CORS configuration** - Replace wildcard CORS with explicit allowed origins. Remove credentials: true from wildcard configurations.

· **Rotate all hardcoded credentials** - Move all passwords, API keys, and database URLs to environment variables. Remove .env files from version control. Rotate all exposed credentials immediately.

· **Sanitize error responses** - Implement global error handlers that return generic error messages to clients while logging full details server-side.

· **Fix login endpoint** - The alo-labs login endpoint must properly validate passwords using bcrypt or argon2 hashing.

Phase 2 - This Sprint (Weeks 2–3)

Objective: Address all High severity findings and improve input handling.

· **Add input validation** - Implement Zod or Joi schemas for all API endpoints. Validate request bodies, query parameters, and path parameters.

· **Implement rate limiting** - Add express-rate-limit or similar to prevent abuse and brute-force attacks on authentication endpoints.

· **Remove dead code** - Delete the 14-file dead modules directory (lab2), 164-line app-store.ts (lab), and 67KB seed.mjs (lab).

· **Fix TypeScript configuration** - Enable noImplicitAny, remove ignoreBuildErrors: true, and enable strict mode. Fix all resulting type errors.

· **Add security headers** - Implement Helmet middleware for Express; configure Next.js security headers.

· **Fix token generation** - Replace Math.random() with crypto.randomBytes() for session token generation.

Phase 3 - Next Sprint (Weeks 4–5)

Objective: Resolve Medium severity findings, improve architecture, and optimize performance.

· **Cleanup unused dependencies** - Audit and remove all unused packages from package.json across all projects.

· **Fix Prisma schema design** - Add unique constraints, change tags from String to Json, add onDelete cascades.

· **Fix performance issues** - Memoize getStatusCounts(), add AbortController to fetch calls, replace synchronous file I/O.

· **Fix data persistence** - Make notes pinning and settings persistent. Replace Math.random() dashboard data with real aggregations.

· **Split monolithic components** - Break up the 7,938-line App.tsx and other oversized components.

· **Fix Toast bugs** - Correct the 11.5-day dismiss delay and useEffect dependency issue.

Phase 4 - Ongoing

Objective: Establish sustainable development practices and prevent regression.

· **Add testing infrastructure** - Implement unit tests (Vitest/Jest), integration tests, and end-to-end tests (Playwright). Target minimum 70% coverage.

· **Create documentation** - Write API documentation (OpenAPI/Swagger), architecture decision records, and onboarding guides.

· **Setup CI/CD pipeline** - Configure automated testing, linting, type-checking, and security scanning (Snyk/Dependabot) on every pull request.

· **Implement monitoring** - Add application monitoring (Sentry), structured logging (Pino/Winston), and health check endpoints.

· **Establish security review process** - Require security review for all changes to authentication, authorization, and data handling code.

7. Conclusion

All three audited projects are in an early prototype or development stage. **None are production-ready** in their current state. The most alarming finding is the complete absence of server-side authentication across the entire portfolio - a fundamental security requirement for any web application handling user data.

The common patterns observed - AI-generated code with security as an afterthought, disabled type checking, no input validation, hardcoded credentials, and missing tests - suggest a need for a fundamental shift in the development approach. While AI-assisted development can accelerate prototyping, it must be paired with proper security review, testing, and quality assurance processes.

We strongly recommend adopting **security-first development practices**: implement authentication and authorization before adding features, enable all compiler strictness options, add input validation to every endpoint, and establish automated testing from the start. The phased remediation roadmap in Section 6 provides a prioritized path to bringing these projects to a production-viable security posture.

> **Key Takeaway:** 12 Critical, 21 High, and 30 Medium severity findings were identified across 3 projects. Immediate action is required on authentication, CORS, and credential management before any production deployment can be considered.